CPP Chapter.12 SQLite Tutorial

Contents

CPP Chapter.12 SQLite Tutorial 1
SQLite and C++ 1
Setup the Project: SQLite with C++ 2
Tutorial 1: Connect to an SQLite Database 2
SQLite DataTypes 4
Creating Tables 4
Tutorial 2: Create a Table 5
sqlite3_exec() 5
Tutorial 3: Insert and Select Records 7
Selecting Data 8
Tutorial 4: Update Records 11
Tutorial 6: Delete Records 12
Assignment Submission 13

Time required: 90 minutes

- Comment each line of code as show in the tutorials and other code examples.
- Follow all directions carefully and accurately.
- Think of the directions as minimum requirements.

SQLite and C++

SQLite3 is the most-used SQL implementation in the world. It is a self-contained C library included in everything from portable devices to web browsers.

SQLite is a self-contained, file-based SQL database. SQLite does not come with C++. We will install a C library to work with SQLite.

Setup the Project: SQLite with C++

We need a C library to work with SQLite database. Download the sqlite-amalgamation(versionnumber).zip file from:

https://www.sqlite.org/download.html

Extract the contents of the ZIP file into your project folder. Only store this program in this folder. The compile batch will attempt to compile every file in the folder.

Tutorial 1: Connect to an SQLite Database

Please keep copies of each tutorial as you go in case something goes wrong. Use the name shown in the tutorial screenshot.

1. In your project folder, create a C++ file named: CreateTable.cpp
2. Insert the following code to create and/or connect to a database.

```cpp
/**
 * Filename: Connect.cpp
 * Written by:
 * Written on:
 * Description: Connect to or create an SQLite3 database
 */

#include <iostream>
#include "sqlite3.h"

int main()
{
    // Pointer to SQLite connection
    sqlite3 *db;

    // Save the connection result
    int dbOpen = 0;

    // If there is a database file, open it,
    // otherwise create a new database file
    dbOpen = sqlite3_open("data.db", &db);

    // Test if there was an error
    if (dbOpen)
    {
        std::cout << "DB Open Error: " << sqlite3_errmsg(db) << std::endl;
    }
    else
    {
        std::cout << "Opened Database Successfully!" << std::endl;
    }

    // Close the connection
    sqlite3_close(db);

    return (0);
}
```

SQLite3 is an external library. To compile it correctly, it must also be linked to the project.

There are multiple files in this project, including the sqlite3 library files.

3. Create a batch file named **ConnectCompile.bat** with the following code.

```
rem Compile the sqlite3 c library. This only needs to be done once.
rem Place rem in front of the line to not compile it each time.
gcc sqlite3.c -c
rem This line compiles the sqlite3 library and the project file.
g++ Connect.cpp -o Connect.exe sqlite3.o -Isqlite3
pause
```

Run the batch file to compile the program.

Example run:

```
Opened Database Successfully!
```

4. You should see **data.db** in your project folder.

SQLite DataTypes

- **NULL:** The value is a NULL value.
- **INTEGER:** Store a whole number.
- **REAL:** Floating-point value, for example, 3.14, the value of PI.
- **TEXT:** A text string. TEXT value stored using UTF-8, UTF-16BE or UTF-16LE encoding.
- **BLOB:** The value is a blob of data, i.e., binary data. It is used to store images and files.

The following Python types convert to SQLite types.

C++ Types	SQLite Types
None	NULL
int	INTEGER
float	REAL
std::string	TEXT
bytes	BLOB

Creating Tables

In an SQL database, data is stored in tables. Tables define a set of columns and fields, much like a spreadsheet. They contain 0 or more rows with data for each of the defined fields.

Let's create a table named **products** that tracks the following data. This is a data dictionary.

PROD_ID (primary key)	INTEGER
PROD_NAME	TEXT
PROD_PRICE	REAL
PROD_QTY	INTEGER

We will create a **products** table using SQLite3 in C++.

SQL is a scripting language like Python. You can assign SQL code to a string variable.

Tutorial 2: Create a Table

Working with SQLite3 includes three main functions: sqlite3_open(), sqlite3_exec(), and sqlite3_close().

In the previous part, the functions sqlite3open() and sqlite3_close() were used to open and close the connection.

To execute SQL, the function sqlite3_exec() is used.

---

sqlite3_exec()

Calls to sqlite3_exec() use five parameters:

- Database connection (SQLite3 pointer)
- SQL to run
- Callback function
- First argument to callback function
- Address of where to write error messages

Because SQL commands like SELECT can return multiple results, the callback function is used to act on them. The fourth parameter is optional when working with functions that may not need it.

To create a table within the existing database, the CREATE TABLE keywords are used in SQL.

Let's create a table.

```cpp
/**
 * Filename: CreateTable.cpp
 * Written by:
 * Written on:
 * Description: Create a SQLite3 table
 */

#include <iostream>
#include <string>
#include "sqlite3.h"

// callback function for SQL operations
int callback(void *NotUsed, int argc, char **argv, char **azColName)
{
    // Return successful
    return 0;
}

int main()
{
    // Pointer to SQLite connection
    sqlite3 *db;
    // Save any error messages
    char *zErrMsg = 0;
    // Save the connection result
    int dbOpen = 0;
    // Variable to store SQL commands
    std::string sql;
    // Save the result of opening the file
    // If there is a database file, open it,
    // otherwise create a new database file
    dbOpen = sqlite3_open("data.db", &db);
```

```
    // Was there an error opening the database file?
    if (dbOpen)
    {
        // Show an error message
        std::cout << "DB Error: " << sqlite3_errmsg(db) << std::endl;
        // Close the connection
        sqlite3_close(db);
        // Return an error
        return (1);
    }
    std::cout << "Opened Database Successfully!" << std::endl;

    // ------------------------ CREATE TABLE ------------------------------//
    // SQL string to create a table
    sql = "CREATE TABLE PRODUCTS ("
          "PROD_ID INT PRIMARY KEY NOT NULL,"
          "PROD_NAME      TEXT    NOT NULL,"
          "PROD_PRICE     REAL,"
          "PROD_QTY       INTEGER)";

    // Run the SQL (convert the string to a C-String with c_str() )
    dbOpen = sqlite3_exec(db, sql.c_str(), callback, 0, &zErrMsg);
    std::cout << "Product Table Created!" << std::endl;

    // Close the SQL connection
    sqlite3_close(db);
    return 0;
}
```

Example run:

```
Opened Database Successfully!
Product Table Created!
```

Tutorial 3: Insert and Select Records

We have a created a table. This is the SQL for inserting records

```
"INSERT INTO PRODUCTS VALUES (1, 'Minecraft', 'Building game', 15.99, 1);
"INSERT INTO PRODUCTS VALUES (2, 'Tetris', 'Sort blocks', 12.99, 23);"
"INSERT INTO PRODUCTS VALUES (3, 'Pong', 'Paddle wars', 2.99, 6);"
"INSERT INTO PRODUCTS VALUES (4, 'Guessing Game', 'What number?', .99, 200);"
```

- INSERT INTO PRODUCTS specifies that we are inserting into the PRODUCTS table.

- VALUES says that the values are coming up next.
- The values must match the datatypes of the table fields.

```
sql = "INSERT INTO PRODUCTS VALUES "
      "(1, 'Minecraft', 'Building game', 15.99, 1);";
```

Selecting Data

Selecting and then fetching the data from records is simple as inserting them. The execute method uses the SQL command of getting all the data from the table using **"SELECT * from table_name"**. The results we access with a loop.

```
/**
 * Filename: InsertRecords.cpp
 * Written by:
 * Written on:
 * Description: Delete, create table, insert and select records
 */

#include <iostream>
#include <string>
#include "sqlite3.h"

// callback function for SQL operations
int callback(void *NotUsed, int argc, char **argv, char **azColName)
{
    // int argc: holds the number of results
    // (array) azColName: holds each column returned
    // (array) argv: holds each value

    for (int i = 0; i < argc; i++)
    {
        // Show column name, value, and newline
        std::cout << azColName[i] << ": " << argv[i] << std::endl;
    }

    // Insert a newline
    std::cout << std::endl;

    // Return successful
    return 0;
}
```

```
32 int main()
33 {
34     // Pointer to SQLite connection
35     sqlite3 *db;
36     // Save any error messages
37     char *zErrMsg = 0;
38     // Save the result of opening the file
39     int dbOpen;
40     // Save any SQL
41     std::string sql;
42     // Save the result of opening the file
43     dbOpen = sqlite3_open("data.db", &db);
44
45     if (dbOpen)
46     {
47         // Show an error message
48         std::cout << "DB Error: " << sqlite3_errmsg(db) << std::endl;
49         // Close the connection
50         sqlite3_close(db);
51         // Return an error
52
53         return (1);
54     }
55     std::cout << " Database Opened Successfully!" << std::endl;
56     dbOpen = sqlite3_exec(db, sql.c_str(), callback, 0, &zErrMsg);
57
58     // ------------------------ DROP TABLE --------------------------------//
59     sql = "DROP TABLE IF EXISTS PRODUCTS;";
60     std::cout << " Products Table Dropped!" << std::endl;
61
62     // ------------------------ CREATE TABLE ------------------------------//
63     // SQL string to create a table
64     sql = "CREATE TABLE PRODUCTS ("
65           "PROD_ID INT PRIMARY KEY NOT NULL,"
66           "PROD_NAME      TEXT    NOT NULL,"
67           "PROD_DESC      TEXT,"
68           "PROD_PRICE     REAL,"
69           "PROD_QTY       INTEGER)";
70
71     // Run the SQL (convert the string to a C-String with c_str() )
72     dbOpen = sqlite3_exec(db, sql.c_str(), callback, 0, &zErrMsg);
73     std::cout << " Products Table Created!" << std::endl;
```

```
    // ------------------------ INSERT -----------------------------------//
    // Save SQL insert data
    sql = "INSERT INTO PRODUCTS VALUES "
          "(1, 'Minecraft', 'Building game', 15.99, 1);";

    // Run the SQL (convert the string to a C-String with c_str() )
    dbOpen = sqlite3_exec(db, sql.c_str(), callback, 0, &zErrMsg);

    // Save SQL insert data
    sql = "INSERT INTO PRODUCTS VALUES "
          "(2, 'Tetris', 'Sort blocks', 12.99, 23);";

    // Run the SQL (convert the string to a C-String with c_str() )
    dbOpen = sqlite3_exec(db, sql.c_str(), callback, 0, &zErrMsg);

    // Save SQL insert data
    sql = "INSERT INTO PRODUCTS VALUES "
          "(3, 'Pong', 'Paddle wars', 2.99, 6);";

    // Run the SQL (convert the string to a C-String with c_str() )
    dbOpen = sqlite3_exec(db, sql.c_str(), callback, 0, &zErrMsg);

    // Save SQL insert data
    sql = "INSERT INTO PRODUCTS VALUES "
          "(4, 'Guessing Game', 'What number?', .99, 200);";

    // Run the SQL (convert the string to a C-String with c_str() )
    dbOpen = sqlite3_exec(db, sql.c_str(), callback, 0, &zErrMsg);

    // ------------------------ SELECT -----------------------------------//
    // Save SQL insert data
    sql = "SELECT * FROM 'PRODUCTS';";

    // Run the SQL (convert the string to a C-String with c_str() )
    dbOpen = sqlite3_exec(db, sql.c_str(), callback, 0, &zErrMsg);

    // Close the SQL connection
    sqlite3_close(db);
    return 0;
}
```

You may have to delete your **data.db** file for this to work.

Example run:

```
 Database Opened Successfully!
 Products Table Dropped!
 Products Table Created!
PROD_ID: 1
PROD_NAME: Minecraft
PROD_DESC: Building game
PROD_PRICE: 15.99
PROD_QTY: 1

PROD_ID: 2
PROD_NAME: Tetris
PROD_DESC: Sort blocks
PROD_PRICE: 12.99
PROD_QTY: 23

PROD_ID: 3
PROD_NAME: Pong
PROD_DESC: Paddle wars
PROD_PRICE: 2.99
PROD_QTY: 6

PROD_ID: 4
PROD_NAME: Guessing Game
PROD_DESC: What number?
PROD_PRICE: 0.99
PROD_QTY: 200
```

Tutorial 4: Update Records

Rows in an SQLite database can be modified using UPDATE SQL statement. We can update single columns as well as multiple columns.

This is the general form to update existing table data.

```
UPDATE TABLE_NAME SET column1 = value1, column2 = value2 WHERE condition;
```

In the above syntax, the SET statement is used to set new values to the column, the WHERE clause is used to select the rows for which the columns are needed to be updated.

Minecraft has a price change. We can change Minecraft's row in the PRODUCT table to reflect this change:

1. We issue an UPDATE SQL statement to change the PROD_PRICE of Minecraft to its new value of 19.99.
2. The WHERE clause in the UPDATE statement ensures we only change the value of PROD_PRICE if a row has PROD_ID = 1.

Add the following code to update our products data. Notice that we are running 2 SQL statements in a single command.

```
    // ------------------------ UPDATE ------------------------------------//
    sql = "UPDATE PRODUCTS SET PROD_PRICE = 19.99 WHERE PROD_ID = 1;"
          "SELECT * FROM 'PRODUCTS';";
    dbOpen = sqlite3_exec(db, sql.c_str(), callback, 0, &zErrMsg);

    // Close the SQL connection
    sqlite3_close(db);
    return 0;
}
```

You may have to delete your **data.db** file for this to work.

Example run. The example only display the changed record.

```
PROD_ID: 1
PROD_NAME: Minecraft
PROD_DESC: Building game
PROD_PRICE: 19.99
PROD_QTY: 1
```

Tutorial 6: Delete Records

```
DELETE FROM TABLE_NAME [WHERE Clause]
```

The project is almost done. Add the following code to finish the project.

```
    // ------------------------ UPDATE ------------------------------------//
    sql = "DELETE FROM PRODUCTS WHERE PROD_ID = 4;"
          "SELECT * FROM 'PRODUCTS';";
    dbOpen = sqlite3_exec(db, sql.c_str(), callback, 0, &zErrMsg);

    // Close the SQL connection
    sqlite3_close(db);
    return 0;
}
```

You may have to delete your **data.db** file for this to work.

Example run:

Record 4 will be deleted.

Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the program.
3. Submit in Blackboard.